: BUBBULE SORT :

Left side, in the same arrow-bullet style as your "What is Sorting?" slide:

➢ What is Bubble Sort?

- A simple comparison-based sorting algorithm.
- It repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.
- After each pass, the largest element "bubbles up" to its correct position at the end.

➢ Key Points

- Needs n − 1 passes for n elements (at most).
- In-place: yes. Stable: yes.
- Simple to understand, but slow for large data

Complexity

- **Best Case:** O(n) — when the array is already sorted, with an optimized version.
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **In-place:** Yes, uses very little extra memory.
- **Stable:** Yes, equal elements keep their relative order.

Key Point: In each pass, the largest unsorted element moves to its correct position.

IN RIGHT SIDE IN BOX WRITE THIS :

Algorithm: Bubble Sort

```
BUBBLESORT(arr, n)

1. For i = 0 to n - 2:
2.     For j = 0 to n - i - 2:
3.         If arr[j] > arr[j + 1]:
4.             Swap arr[j] and arr[j + 1].
5. Stop.
```

In simple words: Compare adjacent elements and swap them if they are in the wrong order. After each pass, the largest unsorted element moves to the end. Repeat until the array is sorted.

#include <stdio.h>

/* Bubble Sort: sorts arr[] in ascending order */ void bubbleSort(int arr[], int n) { int i, j, temp, swapped;

```
for (i = 0; i < n - 1; i++)
{
    swapped = 0;

    /* last i elements are already in place */
    for (j = 0; j < n - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            /* swap adjacent elements */
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
            swapped = 1;
        }
    }

    /* no swaps in this pass -> array is already sorted */
    if (swapped == 0)
        break;
}
```

}

/* Print all elements of the array */ void printArray(int arr[], int n) { int i; for (i = 0; i < n; i++) printf("%d ", arr[i]); printf("\n"); }

int main() { int arr[] = {5, 3, 8, 4, 2}; int n = sizeof(arr) / sizeof(arr[0]);

```
printf("Before sorting: ");
printArray(arr, n);

bubbleSort(arr, n);

printf("After sorting:  ");
printArray(arr, n);

return 0;
```

}

Write below example as it is format all in slide in box okay ?

Writing

Bubble Sort — Step-by-Step Example

Sort in Ascending Order: `5, 3, 8, 2, 4`

Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.

Pass	Steps	Array after Pass
1	(5,3) swap → (5,8) no swap → (8,2) swap → (8,4) swap	**3, 5, 2, 4, 8**
2	(3,5) no swap → (5,2) swap → (5,4) swap	**3, 2, 4, 5, 8**
3	(3,2) swap → (3,4) no swap	**2, 3, 4, 5, 8**
4	(2,3) no swap	**2, 3, 4, 5, 8**

Sorted Array: `2, 3, 4, 5, 8`

Key Point: After each pass, the largest unsorted element moves to its correct position.

Quick sort

Left side — Theory and Steps

➢ What is Quick Sort?

- A fast, comparison-based sorting algorithm.
- It selects an element called the **pivot** and divides the array into two parts.
- Elements smaller than the pivot go to the left, and larger elements go to the right.
- The same process is repeated for both parts until the array is sorted.

➢ Key Points

- Uses the **divide-and-conquer** technique.
- In-place: generally yes, for standard implementations.
- Stable: no, in its usual implementation.
- Usually fast for large datasets.

Complexity

- **Best/Average Case:** O(n log n)
- **Worst Case:** O(n^2)

- **In-place:** Usually yes, apart from recursion stack space.
- **Stable:** No, in its standard implementation.

Key Point: Quick Sort divides the array around a pivot and sorts the smaller parts recursively.

Algorithm: Quick Sort (Using Pivot)

```
QUICKSORT(arr, low, high)

1. If low < high, then
2.     Select the last element as pivot.
3.     Set i = low - 1.
4.     For j = low to high - 1:
5.         If arr[j] < pivot:
6.             i = i + 1
7.             Swap arr[i] and arr[j]
8.     Swap arr[i + 1] and arr[high].
9.     Set p = i + 1.
10.    QUICKSORT(arr, low, p - 1).
11.    QUICKSORT(arr, p + 1, high).
12. Stop.
```

C Program: Quick Sort

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high)
{
    int i, j, pivot, temp;

    if (low < high)
    {
        pivot = arr[high];
        i = low - 1;

        for (j = low; j < high; j++)
        {
            if (arr[j] < pivot)
            {
                i++;
```

```
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int p = i + 1;

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Sample Output

```
Enter number of elements: 5
Enter array elements: 5 3 8 2 4
Sorted array: 2 3 4 5 8
```

Sample example:

Writing

Quick Sort — Step-by-Step Example

Sort in Ascending Order: 5, 3, 8, 2, 4

Quick Sort selects a pivot, places smaller elements on its left and larger elements on its right, and repeats the process for the remaining parts.

Step	Operation	Array
1	Select last element **4** as pivot.	5, 3, 8, 2, 4
2	Move 3 and 2 to the left of pivot 4.	3, 2, 4, 5, 8
3	Sort left part **(3, 2)** using pivot 2.	2, 3, 4, 5, 8
4	Sort right part **(5, 8)** using pivot 8.	2, 3, 4, 5, 8

Sorted Array: 2, 3, 4, 5, 8

Key Point: Quick Sort places the pivot in its correct position and then sorts the left and right parts recursively.